
xmldiff Documentation

Lennart Regebro

Apr 05, 2023

Contents

1	Installation	3
2	Command-line Usage	5
2.1	Options	5
2.2	Formatters	5
2.3	Whitespace Handling	5
2.4	Pretty Printing	6
3	Python API	7
3.1	Main diffing API	7
3.2	Unique Attributes	8
3.3	Using Formatters	9
3.4	The Edit Script	10
3.5	The patching API	13
4	Advanced Usage	15
4.1	Diffing Formatted Text	15
4.2	Making a Visual Diff	16
4.3	Performance Options	17
5	Contributing to <code>xmldiff</code>	19
5.1	Setting Up a Development Environment	19
5.2	Testing	20
5.3	Pull Requests	20
5.4	Code Quality and Conventions	20
5.5	Documentation	20
5.6	Implementation Details	21
6	Indices and tables	23
	Index	25

`xmldiff` is a library and a command-line utility for making diffs out of XML. This may seem like something that doesn't need a dedicated utility, but change detection in hierarchical data is very different from change detection in flat data. XML type formats are also not only used for computer readable data, it is also often used as a format for hierarchical data that can be rendered into human readable formats. A traditional diff on such a format would tell you line by line the differences, but this would not be readable by a human. This library provides tools to make human readable diffs in those situations.

Contents:

CHAPTER 1

Installation

`xmldiff` is a standard Python package and can be installed in all the ways Python packages normally can be installed. The most common way is to use `pip`:

```
pip install xmldiff
```

You can also download the latest version from [The Cheeseshop a.k.a PyPI](#), unpack it with you favourite unpacking tool and then run:

```
python setup.py install
```

That's it, `xmldiff` should now be available for you to use.

Several Unix distributions also include `xmldiff` so you can install it with your distributions package manager. Be aware that currently most distribute an earlier version, typically 0.6.10, which is very different from 2.x, which this documentation is written for. You can check this by running `xmldiff --version`.

CHAPTER 2

Command-line Usage

`xmldiff` is both a command-line tool and a Python library. To use it from the command-line, just run `xmldiff` with two input files:

```
$ xmldiff file1.xml file2.xml
```

There are a few extra options to modify the output, but be aware that not all of the combinations are meaningful, so don't be surprised if you add one and nothing happens.

2.1 Options

2.2 Formatters

You can select different output formats with `xmldiff`, but beware that some formatters may assume certain things about the type of XML.

The included formatters are generic and will work for any type of XML, but may not give you a useful output. If you are using `xmldiff` as a library, you can create your own formatters that is suited for your particular usage of XML.

The `diff` formatter is default and will output a list of edit actions. The `xml` formatter will output XML with differences marked up by tags using the `diff` namespace. The `old` formatter is a formatter that gives a list of edit actions in a format similar to `xmldiff` 0.6 or 1.0.

2.3 Whitespace Handling

Formatters are also responsible for whitespace handling, both in parsing and in output.

By default `xmldiff` will strip all whitespace that is between tags, as opposed to inside tags. That whitespace isn't a part of any data and can be ignored. So this XML structure:

```
<data count="1"></data><data count="2"></data>
```

Will be seen as the same document as this:

```
<data count="1"></data>    <data count="2"></data>
```

Because the whitespace is between the tags. However, this structure is different, since the whitespace there occurs inside a tag:

```
<data count="1">    </data><data count="2"></data>
```

By default the xml formatter will normalize this whitespace. You can turn that off with the `--keep-whitespace` argument.

2.4 Pretty Printing

The term “pretty printing” refers to making an output a bit more human readable by structuring it with whitespace. In the case of XML this means inserting ignorable whitespace into the XML, yes, the same in-between whitespace that is ignored by xmldiff when detecting changes between two files.

xmldiff’s xml formatter understands the `--pretty-print` argument and will insert whitespace to make the output more readable.

For example, an XML output that would normally look like this:

```
<document><story>Some    content</story><story><para>This    is    some    simple    text    with
<i>formatting</i>.</para></story></document>
```

Will with the `--pretty-print` argument look like this:

```
<document>
  <story>Some content</story>
  <story>
    <para>This is some simple text with <i>formatting</i>.</para>
  </story>
</document>
```

This means you can actually use xmldiff to reformat XML, by using the xml formatter and passing in the same XML file twice:

```
$ xmldiff -f xml -p uglyfile.xml uglyfile.xml
```

However, if you keep whitespace with `--keep-whitespace` or `-w`, no reformatting will be done.

3.1 Main diffing API

Using `xmldiff` from Python is very easy, you just import and call one of the three main API methods.

```
>>> from xmldiff import main
>>> main.diff_files("../tests/test_data/insert-node.left.html",
...                 "../tests/test_data/insert-node.right.html",
...                 diff_options={'F': 0.5, 'ratio_mode': 'fast'})
[UpdateTextIn(node='/body/div[1]', text=None),
 InsertNode(target='/body/div[1]', tag='p', position=0),
 UpdateTextIn(node='/body/div/p[1]', text='Simple text')]
```

Which one you choose depends on if the XML is contained in files, text strings or `lxml` trees.

- `xmldiff.main.diff_files()` takes as input paths to files, or file streams.
- `xmldiff.main.diff_texts()` takes as input Unicode strings.
- `xmldiff.main.diff_trees()` takes as input `lxml` trees.

The arguments to these functions are the same:

3.1.1 Parameters

left: The “left”, “old” or “from” XML. The diff will show the changes to transform this XML to the “right” XML.

right: The “right”, “new” or “target” XML.

check: Return error code 1 if there are any differences between the files.

diff_options:

A dictionary containing options that will be passed into the `Differ()`: **F:** A value between 0 and 1 that determines how similar two XML nodes must be to match as the same in both trees. Defaults to 0.5.

A higher value requires a smaller difference between two nodes for them to match. Set the value high, and you will see more nodes inserted and deleted instead of being updated. Set the value low, and you will get more updates instead of inserts and deletes.

`uniqueattrs`: A list of XML node attributes that will uniquely identify a node. See [Unique Attributes](#) for more info.

Defaults to `['{http://www.w3.org/XML/1998/namespace}id']`.

`ratio_mode`:

The `ratio_mode` determines how accurately the similarity between two nodes is calculated. The choices are 'accurate', 'fast' and 'faster'. Defaults to 'fast'.

Using 'faster' often results in less optimal edits scripts, in other words, you will have more actions to achieve the same result. Using 'accurate' will be significantly slower, especially if your nodes have long texts or many attributes.

`ignored_attrs`: A list of XML node attributes that will be ignored in comparison.

fast_match: By default `xmldiff` will compare each node from one tree with all nodes from the other tree. It will then pick the one node that matches best as the match, if that match passes the match threshold `F` (see above).

If `fast_match` is true `xmldiff` will first make a faster run, trying to find chains of matching nodes, during which any match better than `F` will count. This significantly cuts down on the time to match nodes, but means that the matches are no longer the best match, only “good enough” matches.

formatter: The formatter to use, see [Using Formatters](#). If no formatter is specified the function will return a list of edit actions, see [The Edit Script](#).

3.1.2 Result

If no formatter is specified the diff functions will return a list of actions. Such a list is called an Edit Script and contains all changes needed to transform the “left” XML into the “right” XML.

If a formatter is specified that formatter determines the result. The included formatters, `diff`, `xml`, and `old` all return a Unicode string.

`xmldiff` is still under rapid development, and no guarantees are done that the output of one version will be the same as the output of any previous version. The actions of the edit script can be in a different order or replaced by equivalent actions depending on the version of `xmldiff`, but if the Edit Script does not correctly transform one XML tree into another, that is regarded as a bug. This means that the output of the `xml` format also may change from version to version. There is no “correct” solution to how that output should look, as the same change can be represented in several different ways.

3.2 Unique Attributes

The `uniqueattrs` argument is a list of strings or `(tag, attribute)` tuples specifying attributes that uniquely identify a node in the document. This is used by the differ when trying to match nodes. If one node in the left tree has a this attribute, the node in the right tree with the same value for that attribute will match, regardless of other attributes, child nodes or text content. Respectively, if the values of the attribute on the nodes in question are different, or if only one of the nodes has this attribute, the nodes will not match regardless of their structural similarity. In case the attribute is a tuple, the attribute match applies only if both nodes have the given tag.

The default is `['{http://www.w3.org/XML/1998/namespace}id']`, which is the `xml:id` attribute. But if your document have other unique identifiers, you can pass them in instead. If you for some reason do not want the differ to look at the `xml:id` attribute, pass in an empty list.

3.3 Using Formatters

By default the diff functions will return an edit script, but if you pass in a formatter the result will be whatever that formatter returns.

The three included formatters all return Unicode strings.

All formatters take two arguments:

normalize This argument determines whitespace normalizing. It can be one of the following values, all defined in `xmldiff.formatting`:

WS_NONE No normalizing

WS_TAGS Normalize whitespace between tags

WS_TEXT Normalize whitespace in text tags (only used by the `XMLFormatter`).

WS_BOTH Both `WS_TAGS` and `WS_TEXT`.

pretty_print This argument determines if the output should be compact (`False`) or readable (`True`). Only the `XMLFormatter` currently uses this parameter, but it's useful enough that it was included in the `BaseFormatter` class, so that all subsequent formatters may use it.

3.3.1 DiffFormatter

class `xmldiff.formatting.DiffFormatter` (*normalize=WS_TAGS, pretty_print=False*)

This formatter is the one used when you specify `-f diff` on the command line. It will return a string with the edit script printed out, one action per line. Each line is enclosed in brackets and consists of a string describing the action, and the actions arguments. This is the output format of `xmldiff 0.6/1.x`, however, the actions and arguments are not the same, so the output is not compatible.

```
>>> from xmldiff import formatting
>>> formatter = formatting.DiffFormatter()
>>> print(main.diff_files("../tests/test_data/insert-node.left.html",
...                       "../tests/test_data/insert-node.right.html",
...                       formatter=formatter))
[update-text, /body/div[1], null]
[insert, /body/div[1], p, 0]
[update-text, /body/div/p[1], "Simple text"]
```

3.3.2 XmlDiffFormatter

class `xmldiff.formatting.XmlDiffFormatter` (*normalize=WS_TAGS, pretty_print=False*)

This formatter works like the `DiffFormatter`, but the output format is different and more similar to the `xmldiff` output in versions 0.x and 1.x.

```
>>> from xmldiff import formatting
>>> formatter = formatting.XmlDiffFormatter(normalize=formatting.WS_NONE)
>>> print(main.diff_files("../tests/test_data/insert-node.left.html",
...                       "../tests/test_data/insert-node.right.html",
...                       formatter=formatter))
[update, /body/div[1]/text()[1], "\n    "]
[insert-first, /body/div[1],
<p/>]
```

(continues on next page)

(continued from previous page)

```
[update, /body/div/p[1]/text()[1], "Simple text"]
[update, /body/div/p[1]/text()[2], "\n  "]
```

3.3.3 XMLFormatter

`xmldiff.formatting.XMLFormatter(normalize=WS_NONE, pretty_print=True, text_tags=(), formatting_tags=())`

Parameters

- **text_tags** – A list of XML tags that contain human readable text, ex ('para', 'li')
- **formatting_tags** – A list of XML tags that are tags that change text formatting, ex ('strong', 'i', 'u')

This formatter return XML with tags describing the changes. These tags are designed so they easily can be changed into something that will render nicely, for example with XSLT replacing the tags with the format you need.

```
>>> from xmldiff import formatting
>>> formatter = formatting.XMLFormatter(normalize=formatting.WS_BOTH)
>>> print(main.diff_files("../tests/test_data/insert-node.left.html",
...                       "../tests/test_data/insert-node.right.html",
...                       formatter=formatter))
<body xmlns:diff="http://namespaces.shoobx.com/diff">
  <div id="id">
    <p diff:insert="">Simple text</p>
  </div>
</body>
```

3.4 The Edit Script

The default result of the diffing methods is to return an edit script, which is a list of Python objects called edit actions. Those actions tell you how to turn the “left” tree into the “right” tree.

`xmldiff` has nine different actions. These specify one or two nodes in the XML tree, called `node` or `target`. They are specified with an XPATH expression that will uniquely identify the node.

The other arguments vary depending on the action.

3.4.1 InsertNode(target, tag, position)

The `InsertNode` action means that the node specified in `target` needs a new subnode. `tag` specifies which tag that node should have. The `position` argument specifies which position the new node should have, 0 means that the new node will be inserted as the first child of the target. Note that this is different from XPATH, where the first node is 1. This is for ease of use, since Python is zero-indexed.

Example:

```
>>> left = '<document><node>Content</node></document>'
>>> right = '<document><node>Content</node><newnode/></document>'
>>> main.diff_texts(left, right)
[InsertNode(target='/document[1]', tag='newnode', position=1)]
```

3.4.2 DeleteNode (node)

The DeleteNode action means that the node specified in `node` should be deleted.

Example:

```
>>> left = '<document><node>Content</node></document>'
>>> right = '<document/>'
>>> main.diff_texts(left, right)
[DeleteNode (node='/document/node[1]')]
```

3.4.3 MoveNode (node, target, position)

The MoveNode action means that the node specified in `node` should be moved to be a child under the target node. The `position` argument specifies which position it should have, 0 means that the new node will be inserted as the first child of the target. Note that this is different from XPATH, where the first node is 1. This is for ease of use, since Python is zero-indexed.

If the move is within the same parent, the position can be ambiguous. If you have a child that is in position 1, but should be moved to position 3, that position does not include the node being moved, but signifies the position the node should end up at after the move. When implementing a MoveNode () it is therefore easiest to remove the node from the parent first, and then insert it at the given position.

Example:

```
>>> left = '<document><node>Content</node><movenode/></document>'
>>> right = '<document><movenode/><node>Content</node></document>'
>>> main.diff_texts(left, right)
[MoveNode (node='/document/node[1]', target='/document[1]',
            position=1)]
```

3.4.4 InsertAttrib (node, name, value)

The InsertAttrib action means that the node specified in `node` should get a new attribute. The `name` and `value` arguments specify the name and value of that attribute.

Example:

```
>>> left = '<document><node/></document>'
>>> right = '<document newattr="newvalue"><node/></document>'
>>> main.diff_texts(left, right)
[InsertAttrib (node='/document[1]', name='newattr',
               value='newvalue')]
```

3.4.5 DeleteAttrib (node, name)

The DeleteAttrib action means that an attribute of the node specified in `target` should be deleted. The `name` argument specifies which attribute.

Example:

```
>>> left = '<document newattr="newvalue"><node/></document>'
>>> right = '<document><node/></document>'
>>> main.diff_texts(left, right)
[DeleteAttrib(node='/document[1]', name='newattr')]
```

3.4.6 RenameAttrib(node, oldname, newname)

The `RenameAttrib` action means that an attribute of the node specified in `node` should be renamed. The `oldname` and `newname` arguments specify which attribute and its new name.

Example:

```
>>> left = '<document attrib="value"><node/></document>'
>>> right = '<document newattrib="value"><node/></document>'
>>> main.diff_texts(left, right)
[RenameAttrib(node='/document[1]', oldname='attrib',
              newname='newattrib')]
```

3.4.7 UpdateAttrib(node, name)

The `UpdateAttrib` action means that an attribute of the node specified in `node` should get a new value. The `name` and `value` arguments specify which attribute and its new value.

Example:

```
>>> left = '<document attrib="value"><node/></document>'
>>> right = '<document attrib="newvalue"><node/></document>'
>>> main.diff_texts(left, right)
[UpdateAttrib(node='/document[1]', name='attrib', value='newvalue')]
```

3.4.8 UpdateTextIn (node, name)

The `UpdateTextIn` action means that a text content of the node specified in `node` should get a new value. The `text` argument specifies the new value of that text.

Example:

```
>>> left = '<document><node>Content</node></document>'
>>> right = '<document><node>New Content</node></document>'
>>> main.diff_texts(left, right)
[UpdateTextIn (node='/document/node[1]', text='New Content')]
```

3.4.9 UpdateTextAfter (node, name)

The `UpdateTextAfter` action means that a text that trails the node specified in `node` should get a new value. The `text` argument specifies the new value of that text.

Example:


```
>>> left = '<document><node>Content</node></document>'
>>> right = '<document><node>Content</node>Trailing text</document>'
>>> main.diff_texts(left, right)
[UpdateTextAfter(node='/document/node[1]', text='Trailing text')]
```

3.4.10 InsertComment(target, position, text)

Since comments doesn't have a tag, the normal `InsertNode()` action doesn't work nicely with a comment. Therefore comments get their own insert action. Just like `InsertNode()` it takes a target node and a position. It naturally has no tag but instead has a text argument, as all comments have text and nothing else.

`UpdateTextIn()` and `DeleteNode()` works as normal for comments.

Example:

```
>>> left = '<document><node>Content</node></document>'
>>> right = '<document><!-- A comment --><node>Content</node></document>'
>>> main.diff_texts(left, right)
[InsertComment(target='/document[1]', position=0, text=' A comment ')]
```

3.4.11 InsertNamespace(prefix, uri)

Adds a new namespace to the XML document. You need to have this before adding a node that uses a namespace that is not in the original XML tree.

Example:

```
>>> left = '<document></document>'
>>> right = '<document xmlns:new="http://theuri"></document>'
>>> main.diff_texts(left, right)
[InsertNamespace(prefix='new', uri='http://theuri')]
```

3.4.12 DeleteNamespace(prefix)

Removes a namespace from the XML document. You don't need to handle this, strictly speaking, nothing will break if there is an unused namespace, but *xmldiff* will return this action.

Example:

```
>>> left = '<document xmlns:new="http://theuri"></document>'
>>> right = '<document></document>'
>>> main.diff_texts(left, right)
[DeleteNamespace(prefix='new')]
```

3.5 The patching API

There is also an API to patch files using the diff output:

```
>>> from xmldiff import main
>>> print(main.patch_file("../tests/test_data/insert-node.diff",
...                       "../tests/test_data/insert-node.left.html"))
<body>
  <div id="id">
    <p>Simple text</p>
  </div>
</body>
```

On the same line as for the patch API there are three methods:

- `xmldiff.main.patch_file()` takes as input paths to files, or file streams, and returns a string with the resulting XML.
- `xmldiff.main.patch_text()` takes as input Unicode strings, and returns a string with the resulting XML.
- `xmldiff.main.patch_tree()` takes as input one edit script, (ie a list of actions, see above) and one lxml tree, and returns a patched lxml tree.

They all return a string with the patched XML tree. There are currently no configuration parameters for these commands.

4.1 Diffing Formatted Text

You can write your own formatter that understands your XML format, and therefore can apply some intelligence to the format.

One common use case for this is to have more intelligent text handling. The standard formatters will treat any text as just a value, and the resulting diff will simply replace one value with another:

```
>>> from xmldiff import main, formatting
>>> left = '<body><p>Old Content</p></body>'
>>> right = '<body><p>New Content</p></body>'
>>> main.diff_texts(left, right)
[UpdateTextIn(node='/body/p[1]', text='New Content')]
```

The xml formatter will set tags around the text marking it as inserted or deleted:

```
>>> formatter=formatting.XMLFormatter()
>>>
>>> left = '<body><p>Old Content</p></body>'
>>> right = '<body><p>New Content</p></body>'
>>> result = main.diff_texts(left, right, formatter=formatter)
>>> print(result)
<body xmlns:diff="http://namespaces.shoobx.com/diff">
<p><diff:delete>Old</diff:delete><diff:insert>New</diff:insert> Content</p>
</body>
```

But if your XML format contains text with formats, the output can in some cases be less than useful, especially in the case where formatting is added:

```
>>> left = '<body><p>My Fine Content</p></body>'
>>> right = '<body><p><b>My <i>Fine</i> Content</b></p></body>'
>>> result = main.diff_texts(left, right, formatter=formatter)
>>> print(result)
```

(continues on next page)

(continued from previous page)

```
<body xmlns:diff="http://namespaces.shoobx.com/diff">
  <p diff:insert="">
    <b diff:insert="" diff:rename="p">My <i diff:insert="">Fine</i><diff:insert>_
↪Content</diff:insert></b>
  </p>
  <p diff:delete="">My Fine Content</p>
</body>
```

Notice how the the whole text was inserted with formatting, and the whole unformatted text was deleted. The XML-Formatter supports a better handling of text with the `text_tags` and `formatting_tags` parameters. Here is a simple and incomplete example with some common HTML tags:

```
>>> formatter=formatting.XMLFormatter(
...     text_tags=('p', 'h1', 'h2', 'h3', 'h4', 'h5', 'h6', 'li'),
...     formatting_tags=('b', 'u', 'i', 'strike', 'em', 'super',
...                       'sup', 'sub', 'link', 'a', 'span'))
>>> result = main.diff_texts(left, right, formatter=formatter)
>>> print(result)
<body xmlns:diff="http://namespaces.shoobx.com/diff">
  <p>
    <b diff:insert-formatting="">My <i diff:insert-formatting="">Fine</i> Content</b>
  </p>
</body>
```

This gives a result that flags the `<i>` tag as new formatting. This more compact output is much more useful and easier to transform into a visual output.

4.2 Making a Visual Diff

XML and HTML views will of course ignore all these `diff:` tags and attributes. What we want with the HTML output above is to transform the `diff:insert-formatting` attribute into something that will make the change visible. We can achieve that by applying XSLT before the `render()` method in the formatter. This requires subclassing the formatter:

```
>>> import lxml.etree
>>> XSLT = u'<?xml version="1.0"?>
... <xsl:stylesheet version="1.0"
...   xmlns:diff="http://namespaces.shoobx.com/diff"
...   xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
...
...   <xsl:template match="@diff:insert-formatting">
...     <xsl:attribute name="class">
...       <xsl:value-of select="'insert-formatting'"/>
...     </xsl:attribute>
...   </xsl:template>
...
...   <xsl:template match="diff:delete">
...     <del><xsl:apply-templates /></del>
...   </xsl:template>
...
...   <xsl:template match="diff:insert">
...     <ins><xsl:apply-templates /></ins>
...   </xsl:template>
...   </xsl:stylesheet>
```

(continues on next page)

(continued from previous page)

```

...     <xsl:template match="@* | node()">
...         <xsl:copy>
...             <xsl:apply-templates select="@* | node()" />
...         </xsl:copy>
...     </xsl:template>
... </xsl:stylesheet>'''
>>> XSLT_TEMPLATE = lxml.etree.fromstring(XSLT)
>>> class HTMLFormatter(formatting.XMLFormatter):
...     def render(self, result):
...         transform = lxml.etree.XSLT(XSLT_TEMPLATE)
...         result = transform(result)
...         return super(HTMLFormatter, self).render(result)

```

The XSLT template above of course only handles a few cases, like inserted formatting and insert and delete tags (used below). A more complete XSLT file is included [here](#).

Now use that formatter in the diffing:

```

>>> formatter = HTMLFormatter(
...     text_tags=('p', 'h1', 'h2', 'h3', 'h4', 'h5', 'h6', 'li'),
...     formatting_tags=('b', 'u', 'i', 'strike', 'em', 'super',
...                     'sup', 'sub', 'link', 'a', 'span'))
>>> result = main.diff_texts(left, right, formatter=formatter)
>>> print(result)
<body xmlns:diff="http://namespaces.shoobx.com/diff">
  <p>
    <b class="insert-formatting">My <i class="insert-formatting">Fine</i> Content</b>
  </p>
</body>

```

You can then add into your CSS files classes that make inserted text green, deleted text red with an overstrike, and formatting changes could for example be blue. This makes it easy to see what has been changed in a HTML document.

4.3 Performance Options

The performance options available will not just change the performance, but can also change the result. The result will not necessarily be worse, it will just be less accurate. In some cases the less accurate result might actually be preferable. As an example we take the following HTML codes:

```

>>> left = u"""<html><body>
...   <p>The First paragraph</p>
...   <p>A Second paragraph</p>
...   <p>Last paragraph</p>
... </body></html>"""
>>> right = u"""<html><body>
...   <p>Last paragraph</p>
...   <p>A Second paragraph</p>
...   <p>The First paragraph</p>
... </body></html>"""
>>> result = main.diff_texts(left, right)
>>> result
[MoveNode(node='/html/body/p[1]', target='/html/body[1]', position=2),
 MoveNode(node='/html/body/p[1]', target='/html/body[1]', position=1)]

```

We here see that the differ finds that two paragraphs needs to be moved. Don't be confused that it says `p[1]` in both

cases. That just means to move the first paragraph, and in the second case that first paragraph has already been moved and is now last.

If we format that diff to XML with the XMLFormatter, we get output that marks these paragraphs as deleted and then inserted later.

```
>>> formatter = HTMLFormatter(
...     normalize=formatting.WS_BOTH)
>>> result = main.diff_texts(left, right, formatter=formatter)
>>> print(result)
<html xmlns:diff="http://namespaces.shoobx.com/diff">
  <body>
    <p diff:delete="">The First paragraph</p>
    <p diff:delete="">A Second paragraph</p>
    <p>Last paragraph</p>
    <p diff:insert="">A Second paragraph</p>
    <p diff:insert="">The First paragraph</p>
  </body>
</html>
```

Let's try diffing the same HTML with the fast match algorithm:

```
>>> result = main.diff_texts(left, right,
...     diff_options={'fast_match': True})
>>> result
[UpdateTextIn(node='/html/body/p[1]', text='Last paragraph'),
 UpdateTextIn(node='/html/body/p[3]', text='The First paragraph')]
```

Now we instead got two update actions. This means the resulting HTML is quite different:

```
>>> result = main.diff_texts(left, right,
...     diff_options={'fast_match': True},
...     formatter=formatter)
>>> print(result)
<html xmlns:diff="http://namespaces.shoobx.com/diff">
  <body>
    <p><del>The Fir</del><ins>La</ins>st paragraph</p>
    <p>A Second paragraph</p>
    <p><del>La</del><ins>The Fir</ins>st paragraph</p>
  </body>
</html>
```

The texts are updated instead of deleting and then reinserting the whole paragraphs. This makes the visual output more readable. Also note that the XSLT in this case replaced the `<diff:insert>` and `<diff:delete>` tags with `<ins>` and `<del>` tags.

This is a contrived example, though. If you are using xmldiff to generate a visual diff, you have to experiment with performance flags to find the best combination of speed and output for your case.

Contributing to `xmldiff`

`xmldiff` welcomes your help. Replies and responses may be slow, but don't despair, we will get to you, we will answer your questions and we will review your pull requests, but nobody has "Maintain `xmldiff`" as their job description, so it may take a long time. That's open source.

There are some extremely complex issues deep down in `xmldiff`, but don't let that scare you away, there's easy things to do as well.

5.1 Setting Up a Development Environment

To set up a development environment you need a github account, git, and of course Python with pip installed.

You need to clone the repository, and install the development dependencies:

```
git clone git@github.com:Shoobx/xmldiff.git
cd xmldiff
make devenv
```

You should now be able to test your setup by running a few `make` commands:

```
make test
make check
```

or just:

```
make
```

Which will run the syntax and style checkers and the test suit with coverage.

These should both pass with no errors, and then you are set!

5.2 Testing

xmldiff's tests are written using `unittest` and are discoverable by most test runners. There is also a `test` target in the `make` file. The following test runners/commands are known to work:

- `make test`
- `python setup.py test`
- `pytest` (if you have `pytest` installed)
- `python -m unittest`

There is no support for `tox` to run test under different Python versions. This is because Travis will run all supported versions on pull requests in any case, and having yet another list of supported Python versions to maintain seems unnecessary. You can either create your own `tox.ini` file, or you can install [Spiny](#), which doesn't require any extra configuration in the normal case, and will run the tests on all versions that are defined as supported in `setup.py`.

5.3 Pull Requests

Even if you have write permissions to the repository we discourage pushing changes to master. Make a branch and a pull request, and we'll merge that.

You pull requests should:

- Add a test that fails before the change is made
- Keep test coverage at 100%
- Include an description of the change in `CHANGES.txt`
- Add yourself to the contributors list in `README.txt` if you aren't already there.

5.4 Code Quality and Conventions

xmldiff aims to have 100% test coverage. You run a coverage report with `make coverage`. This will generate a HTML coverage report in `htmlcov/index.html`

We run `flake8` as a part of all Travis test runs, the correct way to run it is `make flake`, as this includes only the files that should be covered.

5.5 Documentation

The documentation is written with `sphinx`. It and any other files using the ReStructuredText format, such as `README's` etc, are using a one line per sub-sentence structure. This is so that adding one word to a paragraph will not cause several lines of changes, as that will make any pull request harder to read.

That means that every sentence and most commas should be followed by a new line, except in cases where this obviously do not make sense, for example when using commas to separate things you list. As a result of this there is no limits on line length, but if a line becomes very long you might consider rewriting it to make it more understandable.

You generate the documentation with a `make` command:

```
cd docs
make html
```


The documentation is hosted on [Read the Docs](https://readthedocs.org/projects/xmldiff/), the official URL is <https://readthedocs.org/projects/xmldiff/>.

5.6 Implementation Details

`xmldiff` is based on “[Change Detection in Hierarchically StructuredS Information](#)” by Sudarshan S. Chawathe, Anand Rajaraman, Hector Garcia-Molina, and Jennifer Widom, 1995. It’s not necessary to read and understand that paper in all it’s details to help with `xmldiff`, but if you want to improve the actual diffing algorithm it is certainly helpful.

I hope to extend this section with an overview of how this library does it’s thing.

CHAPTER 6

Indices and tables

- `genindex`
- `modindex`
- `search`

X

`xmldiff.formatting.DiffFormatter` (*built-in class*), [9](#)

`xmldiff.formatting.XmlDiffFormatter`
(*built-in class*), [9](#)